ПРАКТИЧЕСКАЯ РАБОТА «ОБЪЕКТНАЯ МОДЕЛЬ БРАУЗЕРА (BOM) И WINDOW. РАБОТА С ТАЙМЕРАМИ И ДИАЛОГОВЫМИ ОКНАМИ»

***Цель**: научиться управлять браузерным окружением через BOM, реализовывать таймеры (setTimeout/setInterval) для создания интерактивных элементов (секундомер, обратный отсчёт, автосохранение), работать с диалоговыми окнами и объектами location/history/navigator для решения типичных задач Junior-разработчика.*

Подготовка проекта (как в реальной работе)

1. Создайте папку проекта: `js-practice-05`
2. Откройте папку в VS Code.
3. Чтобы не тратить время на CSS, используйте готовую верстку из файла `index.html` и оформление из файла `style.css`. Основная работа выполняется в файле: `script.js`.
4. Создайте репозиторий GitHub с названием `js-practice-05` и загрузите туда проект.

Структура репозитория

```
js-practice-05/
├── index.html
├── style.css
├── script.js
├── screenshots/
│   └── (ваши скриншоты)
└── README.md
```

В **README.md** кратко опишите:

- Название проекта
- Используемые технологии (HTML, CSS, JS)
- Ответы на контрольные вопросы (смотри соответствующий раздел)

Общие требования

- Эта работа проверяет не только наличие работающего кода, а также и ваше ЛИЧНОЕ ПОНИМАНИЕ.
- Код должен быть аккуратным и понятным.
- Валидация пользовательского ввода обязательна.
- Не используйте `document.write()`.
- Не используйте inline-события в HTML (`onclick="..."`). Все обработчики — через `addEventListener` в script.js.
- Вывод результатов — на страницу (через `innerHTML` / `textContent`), не через `alert()` (кроме случаев, когда это явно требуется в задании).
- Для сравнений используйте `===` и `!==`.
- Весь JavaScript — в script.js. Никакого JS в HTML.
- **Комментарии «ПОЧЕМУ?»:** Возле каждого вызова `setTimeout()`, `setInterval()`, `clearTimeout()`, `clearInterval()` **ОБЯЗАТЕЛЬНО** должен быть комментарий: `// Использую setTimeout, потому что...` или `// clearInterval нужен здесь, так как...`
- **УСТНАЯ ЗАЩИТА:** Преподаватель укажет на любую строку кода. Если вы не сможете объяснить, что делает метод и почему выбран именно он — работа не принимается (0 баллов).
- **ТАБУ:** Запрещено забывать вызывать `clearInterval/clearTimeout` там, где таймер должен останавливаться. Нарушение = отказ от засчитывания вашей работы.

Разметка страницы

Сделайте одну страницу с 4-5 секциями. Минимальный набор элементов (можно расширять):

- Секция «Секундомер»: кнопки «Старт», «Стоп», «Сброс» + блок для отображения времени (00:00:00).
- Секция «Обратный отсчёт»: поле ввода для секунд + кнопки «Запустить», «Остановить», «Сброс» + блок для отображения оставшегося времени.
- Секция «Уведомления»: кнопка «Показать уведомление через 3 сек» + блок для отображения уведомления.
- Секция «Информация о браузере»: кнопка «Показать информацию» + блок для вывода данных о браузере (navigator, screen, location).
- Секция «Автосохранение» (для уровня PRO): текстовое поле + индикатор последнего сохранения.

Задания («BOM, таймеры и диалоги»)

ЗАДАНИЯ ПО УРОВНЯМ СЛОЖНОСТИ

Выберите уровень в зависимости от вашей подготовки:

Уровень	Задания
БАЗА - для тех, кому тяжело (минимальные требования)	Задание 1 — секундомер (setInterval)
	Задание 2 — обратный отсчёт (setInterval)
JUNIOR - стандартный уровень (рекомендуется всем)	Задание 3 — уведомления с задержкой (setTimeout)
	Задание 4 — информация о браузере (BOM)
PRO - для сильных студентов (дополнительные челленджи)	Задание 5 — автосохранение формы (setInterval + localStorage)
	Задание 6 — таймер Pomodoro с историей сессий

УРОВЕНЬ «БАЗА» (Для тех, кому тяжело)

Минимальные требования для зачёта. Фокус на понимании setInterval и clearInterval.

Задание 1. Секундомер (setInterval)

Реализуйте простой секундомер с кнопками управления.

Требования:

- Отображение времени в формате ЧЧ:ММ:СС (например, 00:01:25).
- Кнопка «Старт» — запускает секундомер (используйте setInterval с интервалом 1000 мс).
- Кнопка «Стоп» — останавливает секундомер (используйте clearInterval).
- Кнопка «Сброс» — останавливает секундомер и сбрасывает время на 00:00:00.
- Если секундомер уже запущен, повторное нажатие «Старт» не должно запускать второй таймер (проверка через переменную intervalId).
- Используйте padStart(2, "0") для форматирования чисел (5 → 05).

Подсказка — структура кода:

```
let seconds = 0;
let intervalId = null;

const display = document.getElementById("stopwatch-display");
const startBtn = document.getElementById("stopwatch-start");
const stopBtn = document.getElementById("stopwatch-stop");
const resetBtn = document.getElementById("stopwatch-reset");

function updateDisplay() {
    const hours = Math.floor(seconds / 3600);
    const minutes = Math.floor((seconds % 3600) / 60);
    const secs = seconds % 60;

    display.textContent =
        `${String(hours).padStart(2, "0")}:${String(minutes).padStart(2,
"0")}:${String(secs).padStart(2, "0")}`;
}

startBtn.addEventListener("click", () => {
    // Если таймер уже запущен — не запускать второй раз
    if (intervalId !== null) return;

    // Использую setInterval, потому что: (ваш комментарий)
    intervalId = setInterval(() => {
        seconds++;
        updateDisplay();
    }, 1000);
});

stopBtn.addEventListener("click", () => {
    // Использую clearInterval, потому что: (ваш комментарий)
    clearInterval(intervalId);
    intervalId = null; // Важно! Сбрасываем ID
});

resetBtn.addEventListener("click", () => {
    clearInterval(intervalId);
    intervalId = null;
    seconds = 0;
    updateDisplay();
});
```

Задание 2. Обратный отсчёт (setInterval)

Реализуйте таймер обратного отсчёта.

Требования:

- Поле ввода для количества секунд (например, 60).
- Кнопка «Запустить» — начинает обратный отсчёт от введённого значения до 0.
- Отображение времени в формате MM:CC (например, 01:00 → 00:59 → 00:58...).
- Когда время достигает 00:00 — автоматически остановить таймер (clearInterval) и показать сообщение «Время вышло!» на странице (не через alert).
- Кнопка «Остановить» — останавливает таймер.
- Кнопка «Сброс» — останавливает таймер и очищает отображение.
- Валидация: если пользователь ввёл не число или число ≤ 0 — показать сообщение об ошибке.

Подсказка — структура кода:

```
let timeLeft = 0;
let countdownId = null;

const input = document.getElementById("countdown-input");
const display = document.getElementById("countdown-display");
const startBtn = document.getElementById("countdown-start");
```

```
startBtn.addEventListener("click", () => {
    const inputValue = parseInt(input.value, 10);

    // Валидация
    if (isNaN(inputValue) || inputValue <= 0) {
        display.textContent = "❌ Введите корректное число секунд!";
        return;
    }

    // Если таймер уже запущен — остановить предыдущий
    if (countdownId !== null) {
        clearInterval(countdownId);
    }

    timeLeft = inputValue;

    // Использую setInterval, потому что: (ваш комментарий)
    countdownId = setInterval(() => {
        const minutes = Math.floor(timeLeft / 60);
        const seconds = timeLeft % 60;

        display.textContent = `${String(minutes).padStart(2, "0")}:${String(seconds).padStart(2,
"0")}`;

        timeLeft--;

        // Когда время истекло
        if (timeLeft < 0) {
            clearInterval(countdownId);
            countdownId = null;
            display.textContent = "⏰ Время вышло!";
        }
    }, 1000);
});
```

🟡 УРОВЕНЬ «JUNIOR» (Стандарт)

Рекомендуемый уровень для всех. Фокус на setTimeout и работе с BOM.

Задание 3. Уведомления с задержкой (setTimeout)

Реализуйте систему уведомлений с автоматическим скрытием.

Требования:

- Кнопка «Показать уведомление через 3 сек» — через 3 секунды после нажатия показывает блок с уведомлением (например, «☑ Операция выполнена успешно!»).
- Уведомление должно автоматически скрыться через 5 секунд после появления (используйте setTimeout).
- Добавьте кнопку «Закрыть» внутри уведомления — при нажатии уведомление скрывается немедленно и отменяется автоматическое скрытие (используйте clearTimeout).
- Если пользователь нажимает кнопку показа уведомления повторно до истечения 3 секунд — отменить предыдущий таймер и запустить новый (clearTimeout + новый setTimeout).
- Стилизуйте уведомление (зелёный фон, белый текст, позиция fixed в правом верхнем углу).

Подсказка — структура кода:

```
let showTimerId = null;
let hideTimerId = null;

const showBtn = document.getElementById("notification-show");
const notification = document.getElementById("notification");
```

```
const closeBtn = document.getElementById("notification-close");

showBtn.addEventListener("click", () => {
    // Отменяем предыдущий таймер показа, если он есть
    if (showTimerId !== null) {
        clearTimeout(showTimerId);
    }

    // Использую setTimeout, потому что: (ваш комментарий)
    showTimerId = setTimeout(() => {
        notification.style.display = "block";

        // Автоматически скрыть через 5 секунд
        hideTimerId = setTimeout(() => {
            notification.style.display = "none";
        }, 5000);
    }, 3000);
});

closeBtn.addEventListener("click", () => {
    // Отменяем автоматическое скрытие
    clearTimeout(hideTimerId);
    notification.style.display = "none";
});
```

Задание 4. Информация о браузере (BOM)

Выведите информацию о браузере и устройстве пользователя.

Требования:

- Кнопка «Показать информацию» — выводит на страницу следующие данные:

  1. Текущий URL страницы (location.href)
  2. Протокол (location.protocol) — http: или https:
  3. Домен (location.hostname)
  4. Путь (location.pathname)
  5. Язык браузера (navigator.language)
  6. User Agent (navigator.userAgent) — первые 100 символов
  7. Онлайн/офлайн статус (navigator.onLine) — «🟢 Онлайн» или «🔴 Офлайн»
  8. Разрешение экрана (screen.width x screen.height)
  9. Размер окна браузера (window.innerWidth x window.innerHeight)
  10. Определение устройства: «📱 Мобильное устройство» или «💻 Десктоп» (проверка через регулярное выражение /Android|iPhone|iPad|iPod/i.test(navigator.userAgent))

- Оформите вывод в виде списка или таблицы.
- Добавьте кнопку «Обновить информацию» — при изменении размера окна данные должны обновляться.

Подсказка — структура кода:

```
const infoBtn = document.getElementById("browser-info-btn");
const infoBlock = document.getElementById("browser-info");

function displayBrowserInfo() {
    const isMobile = /Android|iPhone|iPad|iPod/i.test(navigator.userAgent);
    const deviceType = isMobile ? "📱 Мобильное устройство" : "💻 Десктоп";
    const onlineStatus = navigator.onLine ? "🟢 Онлайн" : "🔴 Офлайн";

    const info = `
        <ul>
            <li><strong>URL:</strong> ${location.href}</li>
```

```
            <li><strong>Протокол:</strong> ${location.protocol}</li>
            <li><strong>Домен:</strong> ${location.hostname}</li>
            <li><strong>Путь:</strong> ${location.pathname}</li>
            <li><strong>Язык:</strong> ${navigator.language}</li>
            <li><strong>User Agent:</strong> ${navigator.userAgent.substring(0, 100)}...</li>
            <li><strong>Статус:</strong> ${onlineStatus}</li>
            <li><strong>Разрешение экрана:</strong> ${screen.width} x ${screen.height}</li>
            <li><strong>Размер окна:</strong> ${window.innerWidth} x ${window.innerHeight}</li>
            <li><strong>Устройство:</strong> ${deviceType}</li>
        </ul>
    `;

    infoBlock.innerHTML = info;
}

infoBtn.addEventListener("click", displayBrowserInfo);
```

💬 ВОПРОСЫ ДЛЯ ИНТЕРВЬЮ (В README.md)

1. В чём разница между `setTimeout()` и `setInterval()`? Когда использовать каждый?
2. Что произойдёт, если не вызвать `clearInterval()`? Какие проблемы это может вызвать?
3. Что такое BOM? Назовите основные объекты BOM и их назначение.
4. Как узнать, с мобильного ли устройства зашёл пользователь? Приведите пример кода.
5. Почему `alert()`, `confirm()` и `prompt()` считаются устаревшими? Какие у них недостатки?
6. Что делает метод `padStart()`? Зачем он нужен при работе с таймерами?
7. Как сохранить состояние таймера в `localStorage`, чтобы при перезагрузке страницы таймер продолжал работать?
8. Что такое `location.href`? Как перенаправить пользователя на другую страницу через JavaScript?

Скриншоты (папка screenshots/)

- Секундомер в работе (время 00:01:25).
- Обратный отсчёт: начальное состояние и момент «Время вышло!».
- Уведомление, показанное через 3 секунды.
- Блок с информацией о браузере (все 10 пунктов).
- Автосохранение формы: индикатор последнего сохранения.
- Таймер Pomodoro: отображение времени и счётчик сессий.
- Консоль с выводом данных из localStorage (для Задания 5 и 6).

Критерии оценки

- Проект запускается (Live Server), нет ошибок в Console при обычном сценарии.
- Секундомер работает корректно: старт, стоп, сброс, защита от двойного запуска.
- Обратный отсчёт работает корректно: валидация ввода, автоматическая остановка при 00:00.
- Уведомления: задержка показа, автоматическое скрытие, отмена таймеров.
- Информация о браузере: все 10 пунктов выводятся корректно.
- Автосохранение: работает каждые 10 секунд, восстанавливается при загрузке, можно остановить/возобновить.
- Pomodoro: таймер работает, сессии сохраняются в localStorage, история отображается.

- В коде используются `setTimeout, setInterval, clearTimeout, clearInterval` — по назначению, с комментариями.
- Репозиторий оформлен: структура, README с ответами, папка screenshots.

Примеры для проверки

Действие	Ожидаемый результат
Нажать «Старт» на секундомере	Время начинает идти: 00:00:01, 00:00:02...
Нажать «Стоп» на секундомере	Время останавливается на текущем значении
Нажать «Старт» дважды подряд	Второй таймер НЕ запускается (проверка через intervalId)
Ввести 60 в обратный отсчёт и запустить	Отображается 01:00 → 00:59 → 00:58...
Обратный отсчёт достигает 00:00	Таймер останавливается, показывается «Время вышло!»
Нажать «Показать уведомление через 3 сек»	Через 3 секунды появляется уведомление, через 5 секунд скрывается
Нажать «Закрыть» в уведомлении	Уведомление скрывается немедленно
Нажать «Показать информацию о браузере»	Выводятся все 10 пунктов информации
Ввести текст в автосохранение, подождать 10 сек	Индикатор обновляется: «Последнее сохранение: 18:45:32»
Перезагрузить страницу после автосохранения	Текст восстанавливается из localStorage
Запустить Pomodoro, дождаться окончания	Показывается уведомление, счётчик сессий увеличивается на 1

Самостоятельная работа (необязательно, но желательно)

УРОВЕНЬ «PRO» (Для сильных студентов)

Дополнительные баллы. Фокус на комбинировании таймеров с localStorage и реальными сценариями.

Задание 5. Автосохранение формы (setInterval + localStorage)

Реализуйте автосохранение текста в форме (как в Gmail или Google Docs).

Требования:

- Текстовое поле <textarea> для ввода текста.
- Автоматическое сохранение текста в localStorage каждые 10 секунд (используйте setInterval).
- Индикатор последнего сохранения: «Последнее сохранение: 18:45:32» (обновляется при каждом сохранении).
- При загрузке страницы — автоматически восстановить текст из localStorage (если он есть).
- Кнопка «Очистить черновик» — удаляет текст из localStorage и очищает поле.
- Кнопка «Остановить автосохранение» / «Возобновить автосохранение» — управление таймером (clearInterval / setInterval).
- Если пользователь не вводил текст (поле пустое) — не сохранять в localStorage.

Подсказка — структура кода:

```
let autosaveId = null;
```

```
const textarea = document.getElementById("autosave-textarea");
const indicator = document.getElementById("autosave-indicator");
const toggleBtn = document.getElementById("autosave-toggle");
const clearBtn = document.getElementById("autosave-clear");

function saveToLocalStorage() {
    const text = textarea.value.trim();

    if (text === "") {
        return; // Не сохраняем пустой текст
    }

    localStorage.setItem("draft", text);

    const now = new Date();
    const timeString = now.toLocaleTimeString("ru-RU");
    indicator.textContent = `Последнее сохранение: ${timeString}`;
}

function startAutosave() {
    // Использую setInterval, потому что: (ваш комментарий)
    autosaveId = setInterval(saveToLocalStorage, 10000); // 10 секунд
    toggleBtn.textContent = "Остановить автосохранение";
}

function stopAutosave() {
    clearInterval(autosaveId);
    autosaveId = null;
    toggleBtn.textContent = "Возобновить автосохранение";
}

// При загрузке страницы — восстановить черновик
window.addEventListener("load", () => {
    const savedDraft = localStorage.getItem("draft");
    if (savedDraft) {
        textarea.value = savedDraft;
        indicator.textContent = "Черновик восстановлен";
    }
    startAutosave();
});

toggleBtn.addEventListener("click", () => {
    if (autosaveId === null) {
        startAutosave();
    } else {
        stopAutosave();
    }
});

clearBtn.addEventListener("click", () => {
    localStorage.removeItem("draft");
    textarea.value = "";
    indicator.textContent = "Черновик удалён";
});
```

Задание 6. Таймер Pomodoro с историей сессий

Реализуйте полноценный таймер Pomodoro (техника управления временем).

Требования:

- Таймер на 25 минут (для тестирования можно сократить до 25 секунд).
- Кнопки: «Старт», «Пауза», «Сброс».
- Отображение времени в формате ММ:СС.
- Когда время истекает — показать уведомление « ⏰ Время на перерыв!» (можно использовать alert или кастомное уведомление).
- Счётчик завершённых «помидоров» (сессий) — увеличивается при каждом завершении таймера.

- История сессий: сохранять в localStorage массив объектов с данными о каждой сессии: { date: "2026-03-05", time: "18:45", duration: 25 }.
- Кнопка «Показать историю» — выводит список всех завершённых сессий.
- Кнопка «Очистить историю» — удаляет историю из localStorage.
- Сохранение состояния таймера в localStorage — при перезагрузке страницы таймер продолжает работать с того же места (сохранять оставшееся время и статус «запущен/остановлен»).

Дополнительные челленджи для продвинутых студентов:

1. Добавьте звуковое уведомление при завершении таймера (используйте `<audio>` элемент).
2. Реализуйте «Режим фокусировки»: при запуске Pomodoro блокируйте все остальные кнопки на странице (`disabled`).
3. Добавьте визуальный прогресс-бар для таймеров (используйте `<progress>` элемент или CSS).
4. Реализуйте проверку активности пользователя: если пользователь неактивен 5 минут (нет движения мыши, нажатий клавиш) — показать `confirm()` с вопросом «Вы ещё здесь?».
5. Добавьте экспорт истории Pomodoro в JSON-файл (кнопка «Скачать историю»).

Форма сдачи: Отправьте преподавателю ссылку на GitHub-репозиторий в Slack

Примечание: при необходимости вы можете упростить стили. Оценивается логика, корректность условий, обработка ошибок и аккуратность кода.